

Trabajo Práctico: Algoritmos Greedy

Tema: Optimización Voraz & Problema del Cambio de Monedas | **Modalidad:** Individual / Parejas

Contexto del Problema: Sistema de Peajes Automáticos

Una empresa de tecnología vial te encarga desarrollar el módulo encargado de dar el cambio automático en las cabinas de peaje de una nueva autopista interurbana. Para agilizar el tránsito y evitar embotellamientos, las máquinas deben entregar el cambio utilizando el **menor número total de monedas/billetes posible**. Cada cabina cuenta con un stock **ilimitado** de dinero en efectivo.

Parte 1: Formalización Teórica y Pseudocódigo

1. Componentes del Algoritmo Greedy

Tomando como referencia el esquema general de los algoritmos voraces, identifiquen y definan formalmente los 5 componentes para este problema específico:

- **Conjunto de Candidatos (C):** Colección de denominaciones disponibles.
- **Función de Selección:** Criterio para elegir la moneda en cada paso local.
- **Función de Factibilidad:** Condición que verifica si la moneda elegida no supera el vuelto restante.
- **Función Objetivo:** Métrica a minimizar (cantidad total de elementos entregados).
- **Función Solución:** Condición de parada que determina cuando el cambio se ha completado exactamente.

2. Implementación / Pseudocódigo

Escriban la implementación o pseudocódigo de la función *cambioGreedy(monto, denominaciones)*. Debe retornar el número total de monedas utilizadas y el desglose correspondiente.

```
Funcion cambioGreedy(monto: Entero, D: Arreglo de Enteros) -> Resultado // D presupone
un arreglo ordenado de mayor a menor denominación
solucion = DiccionarioVacio()
monedasTotales = 0 montoRestante = monto i = 0 Mientras montoRestante > 0 y i <
longitud(D) Hacer Si D[i] <= montoRestante Entonces cantidad = montoRestante DIV D[i]
solucion.agregar(D[i], cantidad) monedasTotales = monedasTotales + cantidad
montoRestante = montoRestante MOD D[i] FinSi i = i + 1 FinMientras Retornar
(monedasTotales, solucion) FinFuncion
```

Parte 2: Análisis de Casos de Prueba y Optimalidad

La empresa opera temporalmente en dos regiones con distintos sistemas monetarios:

- **Sistema A (Moneda Actual):** $D_A = [100, 50, 20, 10, 5, 2, 1]$
- **Sistema B (Moneda Especial):** $D_B = [4, 3, 1]$

Consignas de Análisis

1. Simulación Manual y Devolución:

Apliquen el algoritmo Greedy para devolver un cambio de $v = 6$ en el **Sistema B** ($D_B = [4, 3, 1]$).

- ¿Cuál es el resultado (número total y desglose de monedas) proporcionado por el algoritmo Greedy?
- ¿Existe una combinación que requiera *menos monedas* para dar el cambio exacto de \$6? Detállenla.

2. Análisis Crítico de la Teórica Voraz:

- ¿Por qué el algoritmo Greedy garantiza una solución óptima en el **Sistema A**, pero falla en encontrar la solución óptima en el **Sistema B**?
- Expliquen qué propiedad estructural poseen las denominaciones del Sistema A y por qué la falta de "reconsideración de decisiones pasadas" (propiedad miope del algoritmo voraz) perjudica el resultado en el Sistema B.

3. Análisis de Complejidad Temporal:

Determinen la complejidad temporal del algoritmo en el peor de los casos en función del monto v y la cantidad de denominaciones k (consideren si el vector requiere ser ordenado previamente).

Guía de Criterios de Evaluación (Rúbrica)

Criterio	Descripción de Desempeño Esperado
Identificación de Funciones	Diferenciación clara y correcta de los 5 elementos del esquema Greedy (Candidatos, Selección, Factibilidad, Objetivo, Solución).
Correctitud del Algoritmo	Implementación lógica adecuada sin bucles infinitos, con cálculo correcto de residuos y desglose de tipos de moneda.
Detección de Suboptimalidad	Reconocimiento de que para \$6 en Sistema B, Greedy devuelve 3 monedas (4 + 1 + 1) mientras que el óptimo son 2 monedas (3 + 3).
Fundamentación Teórica	Explicación clara de la miopía del enfoque voraz y análisis de la propiedad de subestructura óptima / canonicidad del sistema de monedas.